

□ Redux Toolkit (RTK) — Handwritten Notes

1. What is Redux?

Redux is a state management library.

It is used when many components need to access or modify the same state.

Example: an e-commerce application where Navbar, Product Page and Cart Page all need the same cart state.

Think: Redux provides a central store instead of passing props through many levels.

2. What is Redux Toolkit?

Redux Toolkit (RTK) is the official recommended way to write Redux code.

Old Redux required lots of boilerplate: action types, actions, reducers, store, etc. Redux Toolkit makes Redux much simpler.

3. Why do we need Redux?

React already provides `useState`, `useReducer` and `useContext`. Redux becomes useful when an application has large or complex shared state used by many components.

4. Important Redux Terms

Store — central application state Slice — one feature's state + reducers State — current data

Action — describes what happened Reducer — describes how state changes Dispatch — sends an action Selector — reads state

5. Redux Store

The store contains the application's Redux state.

Example:

```
{
  user: {
    name: "Vijay",
    isLoggedIn: true
  },

  cart: {
    items: [],
    total: 0
  }
}
```

6. Slice

A slice represents one part of the application state.

Example: `userSlice`, `cartSlice`, `productSlice`, `authSlice`.

7. `createSlice()`

`createSlice()` is one of the most important Redux Toolkit functions.

A slice normally contains name, initialState and reducers.

```
const counterSlice = createSlice({
  name: "counter",

  initialState: {
    count: 0
  },

  reducers: {
    increment: (state) => {
      state.count += 1;
    }
  }
});
```

8. Reducer

A reducer describes how the state should change.

Example: increment changes count from 5 to 6.

9. Action

An action tells Redux: "I want this state change to happen."

You normally do not call a reducer directly. You dispatch an action.

10. Dispatch

dispatch() sends an action to Redux.

Think: dispatch = send an instruction to Redux.

11. Selector

useSelector() is used to read data from the Redux Store.

Remember: useSelector → READ dispatch → CHANGE

12. Provider

Provider makes the Redux store available to React components.

Usually it is placed around in main.jsx.

```
<Provider store={store}>
  <App />
</Provider>
```

13. Basic Redux Toolkit Structure

A common project structure:

```
src/
├── app/
│   └── store.js
├── features/
│   └── counter/
│       └── counterSlice.js
├── App.jsx
└── main.jsx
```

14. Create Store

configureStore() creates the Redux store and connects slices/reducers.

```
import { configureStore } from "@reduxjs/toolkit";

export const store = configureStore({
  reducer: {
    counter: counterReducer
  }
});
```

15. Create Slice

createSlice() creates the slice, actions and reducer for a feature.

```
const counterSlice = createSlice({
  name: "counter",

  initialState: {
    count: 0
  },

  reducers: {
    increment: (state) => {
      state.count += 1;
    },
    decrement: (state) => {
      state.count -= 1;
    }
  }
});

export const { increment, decrement } =
  counterSlice.actions;

export default counterSlice.reducer;
```

16. Connect Redux to React

Wrap the application with so components can access Redux.

```
import { Provider } from "react-redux";
import { store } from "../app/store";

<Provider store={store}>
  <App />
</Provider>
```

17. Reading Redux State

Use useSelector() inside a component to read state from the store.

```
const count = useSelector(
  (state) => state.counter.count
);
```

18. Updating Redux State

Use useDispatch() and dispatch(action) to trigger a state change.

```
const dispatch = useDispatch();

dispatch(increment());
```

19. Payload

Payload carries data along with an action.

Example: `dispatch(incrementBy(5))` means `action.payload` is 5.

```
incrementBy: (state, action) => {  
  state.count += action.payload;  
}
```

```
dispatch(incrementBy(5));
```

20. action.payload

`action.payload` contains the data sent with the action.

It can be a number, string, object, array, etc.

```
dispatch(addUser({  
  name: "Vijay",  
  age: 21  
}));
```

```
// action.payload  
// { name: "Vijay", age: 21 }
```

21. Why can Redux Toolkit “mutate” state?

Redux state should normally be immutable. Redux Toolkit uses Immer internally, allowing mutation-style code such as `state.count += 1` while producing an immutable update internally.

22. Redux vs useContext

`useContext` is mainly useful for sharing simple values such as theme, user, language or settings.

Redux Toolkit is suited to more complex shared state such as cart, products, authentication, orders and notifications.

23. Redux Data Flow

Redux follows one-way data flow:

Component → `dispatch(action)` → Action → Reducer → Redux Store → State Updated → Component → UI

```
Component  
↓  
dispatch(action)  
↓  
Action  
↓  
Reducer  
↓  
Redux Store  
↓  
State Updated  
↓  
Component  
↓  
UI
```

24. Multiple Slices

Real applications usually have multiple slices such as auth, user, products, cart and orders.

25. Redux in a MERN Application

For a MERN job portal, Redux can manage auth, jobs, users/profile and other shared application state.

Example flow: Login → API request → success → store user/auth state → Navbar/Profile/Protected pages update.

26. Redux Toolkit + API

Redux Toolkit can manage API-related state such as: data, loading and error.

Typical flow: API request → pending → fulfilled (success) OR rejected (error).

```
{
  data: [],
  loading: false,
  error: null
}
```

Component

↓

dispatch(fetchJobs())

↓

loading = true

↓

API response

↓

data = jobs

loading = false

27. createAsyncThunk

createAsyncThunk is commonly used for asynchronous operations such as GET, POST, PUT and DELETE requests.

Flow: dispatch(fetchJobs()) → pending → API response → fulfilled/rejected.

```
const fetchUsers = createAsyncThunk(
  "users/fetchUsers",
  async () => {
    const response =
      await axios.get("/api/users");

    return response.data;
  }
);
```

28. Redux Toolkit vs useState

useState is best for local component state such as opening a modal. Redux Toolkit is useful when many parts of the application need the same state, such as cart data.

29. Redux Toolkit vs useReducer

useReducer is generally used for component-level state management. Redux Toolkit is used for application-level state management with a centralized store.

Both use the concepts of state + action + reducer.

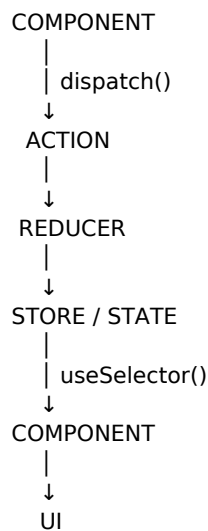
□ Quick Revision

Redux → State management library
Redux Toolkit → Recommended modern Redux approach
Store → Central Redux state
Slice → One feature's state + reducers
Reducer → Changes state
Action → Describes what happened
Dispatch → Sends action
useSelector → Reads state
Provider → Gives React access to Redux store
Payload → Data sent with an action
createAsyncThunk → Handles async logic/API requests

□ Learning Order

1. Why Redux? 2. Store 3. createSlice 4. Reducer 5. Action 6. dispatch() 7. useSelector() 8. Payload 9. Multiple slices 10. createAsyncThunk 11. API + Redux 12. Authentication with Redux

□ Most Important Flow



□ Study tip: Do not memorize the code first. Understand the flow: Component → dispatch → action → reducer → store → selector → component. Once this flow is clear, Redux Toolkit becomes much easier.